

END-TO-END CI/CD PIPELINE USING JENKINS, DOCKER AND AWS EC2

ABSTRACT

Continuous Integration and Continuous Deployment (CI/CD) have become essential practices in modern software development. Manual deployment processes are time-consuming and error-prone. This project aims to automate the software delivery process using Jenkins, Docker, GitHub, Docker Hub, and AWS EC2. The developed system automatically builds, packages, and deploys an application whenever changes are pushed to the GitHub repository. GitHub Webhooks trigger Jenkins, which builds a Docker image, pushes it to Docker Hub, and deploys the latest container to an AWS EC2 instance. This reduces manual intervention and enables rapid and reliable application deployment.

CHAPTER 1: INTRODUCTION

1.1 Overview

Continuous Integration and Continuous Deployment (CI/CD) are software engineering practices that automate application build, testing, and deployment processes. This project implements a complete CI/CD pipeline that integrates GitHub, Jenkins, Docker, Docker Hub, and AWS EC2 to achieve automated application deployment.

1.2 Problem Statement

Traditional software deployment methods involve manual execution of build and deployment tasks, resulting in:

- Increased deployment time
- Human errors
- Inconsistent environments
- Reduced scalability

Therefore, an automated deployment system is required.

1.3 Objectives

- Automate application deployment.
- Integrate GitHub with Jenkins using Webhooks.
- Build Docker images automatically.
- Push Docker images to Docker Hub.
- Deploy containers on AWS EC2.
- Reduce manual effort and deployment time.

CHAPTER 2: LITERATURE REVIEW

Continuous Integration enables developers to integrate code changes frequently. Jenkins provides automation capabilities while Docker ensures portability through containerization. Cloud platforms such as AWS EC2 provide scalable infrastructure for hosting applications. Docker Hub acts as a centralized image repository enabling image distribution and version management.

CHAPTER 3: SYSTEM REQUIREMENTS

Hardware Requirements

Component Specification

Processor	Intel Core i5 or above
RAM	8 GB
Storage	256 GB SSD
Internet	Broadband Connection

Software Requirements

Software	Version
Windows 11	Latest
Git	2.53
Jenkins	2.555.2
Docker Desktop	Latest

Software	Version
GitHub	Cloud
Docker Hub	Cloud
AWS EC2	Ubuntu Instance
PowerShell	Latest

CHAPTER 4: SYSTEM ARCHITECTURE

Developer



GitHub Repository



GitHub Webhook



Jenkins Pipeline



Docker Build



Docker Hub



AWS EC2 Instance



Docker Container



Live Application

CHAPTER 5: IMPLEMENTATION

Step 1: Source Code Management

Application source code is maintained in a GitHub repository.

Repository:

<https://github.com/prats00017/devops-build>

Step 2: GitHub Webhook

GitHub sends POST requests to Jenkins whenever code is pushed to the repository.

Webhook Endpoint:

`http://<jenkins-server>/github-webhook/`

Step 3: Jenkins Pipeline

Pipeline Stages:

Clone Repository

Fetches latest source code from GitHub.

Build Docker Image

Command:

`docker build -t prateek0017/devops-build-dev .`

Docker Login

Authenticates with Docker Hub.

Push Docker Image

Command:

`docker push prateek0017/devops-build-dev`

Deploy to AWS EC2

Jenkins connects to EC2 using SSH and executes:

`docker pull prateek0017/devops-build-dev`

`docker stop devops-build`

`docker rm devops-build`

```
docker run -d -p 80:3000 --name devops-build prateek0017/devops-build-dev
```

CHAPTER 6: PIPELINE WORKFLOW

Git Push



Webhook Trigger



Jenkins Build



Docker Build



Docker Hub Push



SSH to EC2



Docker Deployment



Application Live

CHAPTER 7: RESULTS

The CI/CD pipeline successfully performs:

- Automatic code integration.
- Automatic Docker image creation.
- Docker image storage in Docker Hub.
- Automatic deployment to AWS EC2.
- Continuous delivery with minimal human intervention.

Live Application URL:

<http://43.205.138.8>

CHAPTER 8: ADVANTAGES

- Faster deployments
 - Reduced human errors
 - Continuous integration
 - Scalable architecture
 - Easy rollback and maintenance
 - Platform-independent deployment
-

CHAPTER 9: FUTURE SCOPE

- Kubernetes deployment using EKS.
 - Infrastructure provisioning using Terraform.
 - Monitoring using Prometheus and Grafana.
 - Reverse proxy with Nginx.
 - SSL certificate integration.
 - SonarQube integration for code quality.
 - Blue-Green deployment strategy.
-

CONCLUSION

The project successfully demonstrates an automated CI/CD pipeline using Jenkins, Docker, Docker Hub, and AWS EC2. The system eliminates manual deployment processes and enables continuous delivery with improved efficiency, scalability, and reliability. This implementation follows modern DevOps practices and can be extended further using Kubernetes and monitoring tools.

REFERENCES

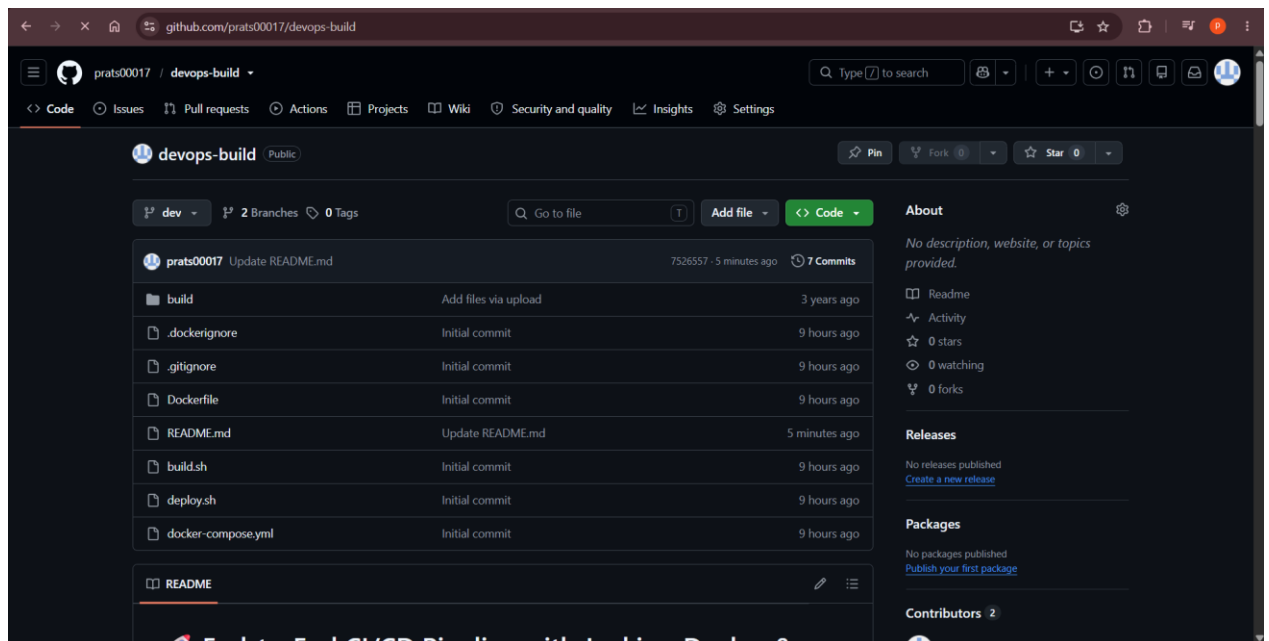
1. Jenkins Documentation
2. Docker Documentation

3. AWS EC2 Documentation
4. GitHub Documentation
5. Docker Hub Documentation
6. DevOps Handbook by Gene Kim
7. Continuous Delivery by Jez Humble and David Farley

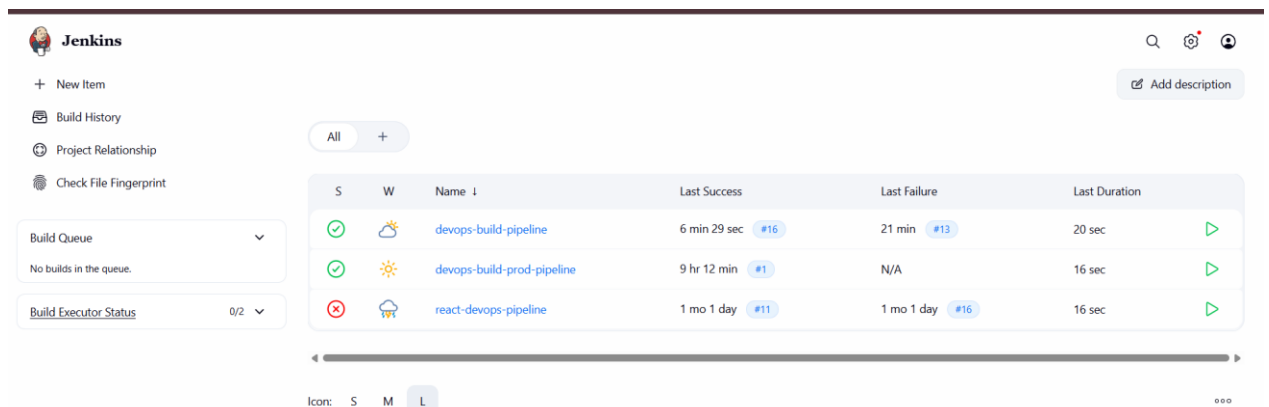
APPENDIX

Screenshots to Include

1. GitHub Repository



2. Jenkins Dashboard



3. Jenkins Pipeline Console Output

Jenkins / devops-build-pipeline

Status

devops-build-pipeline

Add description

Changes

Build Now

Configure

Delete Pipeline

Stages

Rename

Pipeline Syntax

GitHub Hook Log

Builds

Filter

Today

- #16 9:24 pm
- #15 9:17 pm
- #14 9:14 pm
- #13 9:09 pm
- #12 2:56 pm

Permalinks

- Last build (#16), 7 min 23 sec ago
- Last stable build (#16), 7 min 23 sec ago
- Last successful build (#16), 7 min 23 sec ago
- Last failed build (#13), 22 min ago
- Last unsuccessful build (#13), 22 min ago
- Last completed build (#16), 7 min 23 sec ago

Jenkins / devops-build-pipeline / #16

Status

Changes

Console Output

Edit Build Information

Delete build '#16'

Polling Log

Timings

Git Build Data

Pipeline Overview

Restart from Stage

Replay

Pipeline Steps

Workspaces

Previous Build

Console

Download

Copy

View as plain text

```
Started by GitHub push by prateek0017
[Pipeline] Start of Pipeline (hide)
[Pipeline] node
Running on Jenkins in C:\ProgramData\Jenkins\jenkins\workspace\devops-build-pipeline
[Pipeline] {
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Clone Repository)
[Pipeline] git
The recommended git tool is: NONE
No credentials specified
> git.exe rev-parse --resolve-git-dir C:\ProgramData\Jenkins\jenkins\workspace\devops-build-pipeline\.git # timeout=10
Fetching changes from the remote Git repository
> git.exe config remote.origin.url https://github.com/prateek0017/devops-build.git # timeout=10
Fetching upstream changes from https://github.com/prateek0017/devops-build.git
> git.exe --version # timeout=10
> git --version # 'git version 2.53.0.windows.2'
> git.exe fetch --tags --force --progress -- https://github.com/prateek0017/devops-build.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git.exe rev-parse "refs/remotes/origin/dev:[commit]" # timeout=10
Checking out Revision 752655709dfa627f2dd3b64f01c9e9fae47ecce0 (refs/remotes/origin/dev)
> git.exe config core.sparsecheckout # timeout=10
> git.exe checkout -f 752655709dfa627f2dd3b64f01c9e9fae47ecce0 # timeout=10
> git.exe branch -a -v --no-abbrev # timeout=10
> git.exe branch -D dev # timeout=10
```

Jenkins / devops-build-pipeline / #16

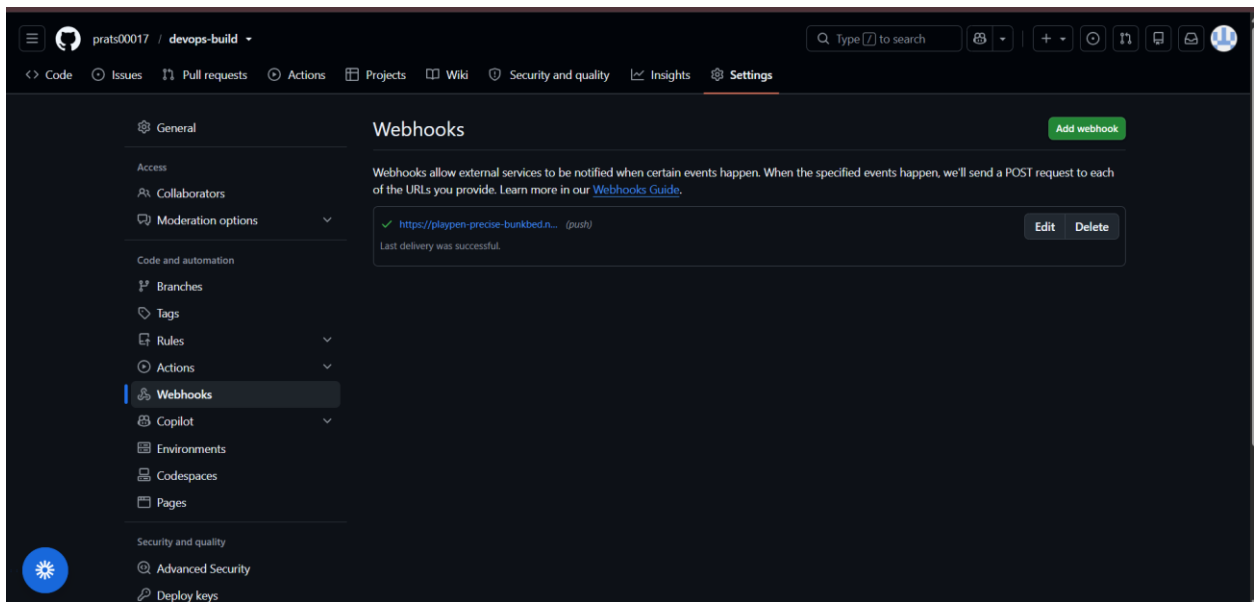
[Pipeline] stage

[Pipeline] { (Deploy to EC2)

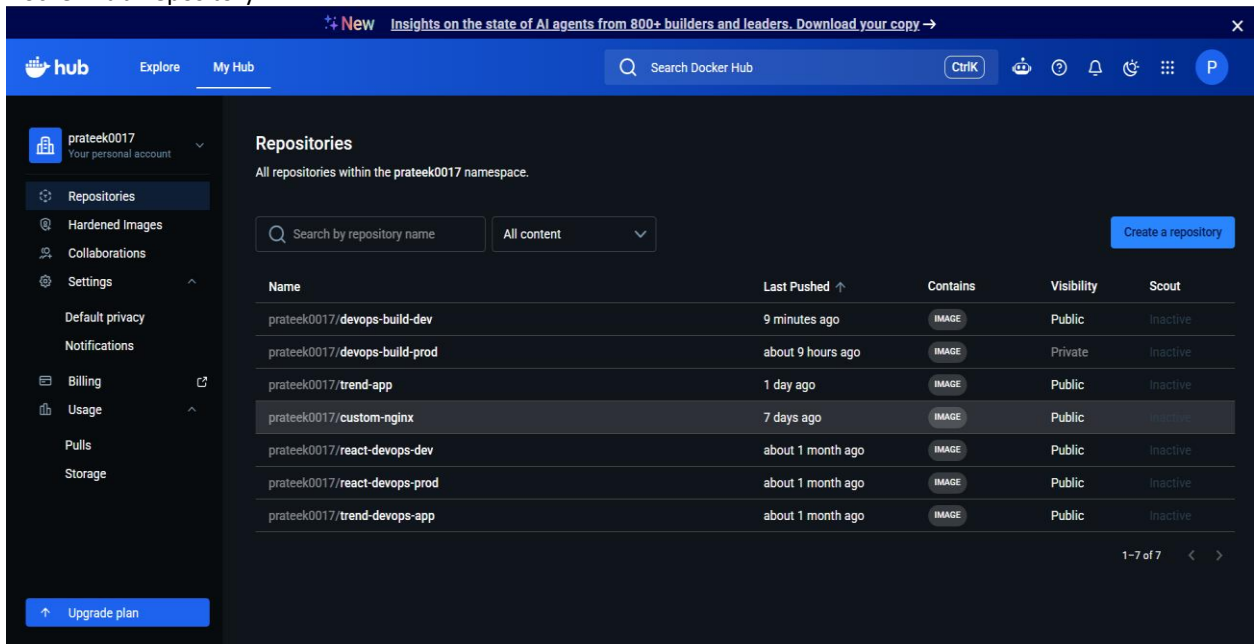
[Pipeline] bat

```
C:\ProgramData\Jenkins\jenkins\workspace\devops-build-pipeline>"C:\Program Files\PuTTY\plink.exe" -batch -hostkey "ssh-ed25519 255
SHA256:sJ0fmg+hlt/0avS50M2gksXpe59a28jGZwKe55zX0xi" -i "C:\Users\prateek\Downloads\devops-key.ppk" ubuntu@43.205.138.8 "docker: pull
prateek0017/devops-build-dev:latest && docker stop devops-build-app || true && docker rm devops-build-app || true && docker run -d --name
devops-build-app -p 80:80 prateek0017/devops-build-dev:latest"
latest: Pulling from prateek0017/devops-build-dev
Digest: sha256:c86476704f977853c64657d580365b6153b97e118c47949d3701e324f76a4083
Status: Downloaded newer image for prateek0017/devops-build-dev:latest
docker.io/prateek0017/devops-build-dev:latest
devops-build-app
devops-build-app
e0e5e03bc933a8fcb5765a3abec24d1a0aaa04987876c196916d52f73c954
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

4. GitHub Webhook Deliveries



5. Docker Hub Repository



Repositories

Hardened Images

Collaborations

Settings

Default privacy

Notifications

Billing

Usage

Pulls

Storage

prateek0017/devops-build-dev

Last pushed 2 minutes ago · Repository size: 26.3 MB · 0 · 98

Add a description

Add a category

General

Tags

Image Management

Collaborators

Webhooks

Settings

Tags

This repository contains 1 tag(s).

Tag	OS	Type	Pulled	Pushed
latest		Image	less than 1 day	2 minutes

See all

Repository overview

An overview describes what your image does and how to run it. It displays in the public view of your repository once you have pushed some content.

Upgrade plan

Add overview

Docker commands

Public view

To push a new tag to this repository:

docker push prateek0017/devops-build-dev:tagname

buildcloud

Build with Docker Build Cloud

Accelerate image build times with access to cloud-based builders and shared cache.

Docker Build Cloud executes builds on optimally-dimensioned cloud infrastructure with dedicated per-organization isolation.

Get faster builds through shared caching across your team, native multi-platform support, and encrypted data transfer - all without managing infrastructure.

Go to Docker Build Cloud

6. AWS EC2 Instance

EC2

Instances

Dashboard

AWS Global View

Events

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Capacity Manager

Images

AMIs

AMI Catalog

Elastic Block Store

Volumes

Instances (1/8) Info

Last updated less than a minute ago

Connect

Instance state

Actions

Launch instances

Saved filter sets

Choose filter set

Find Instance by attribute or tag (case-sensitive)

Instance state = running

Clear filters

	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	F
☐	Kubernetes task	i-08f1b8c9dabf6137e	Running	t3.micro	3/3 checks passed	ap-south-1a	ec2-3-110-186-57.ap-s...	3
☐	Jenkins Task	i-0b15884fd034e63eb	Running	t3.micro	3/3 checks passed	ap-south-1a	ec2-65-0-185-77.ap-so...	6
☐		i-02776d81adaa3baf4	Running	t3.small	3/3 checks passed	ap-south-1a	ec2-15-206-27-255.ap-...	1
☐	jenkins-server	i-0fd4349e32aebf90d	Running	t3.micro	3/3 checks passed	ap-south-1a	-	1
☒	devops-build-...	i-077163f7c61bc2ca7	Running	t3.micro	3/3 checks passed	ap-south-1a	ec2-43-205-138-8.ap-s...	4

i-077163f7c61bc2ca7 (devops-build-server)

Security details

IAM role

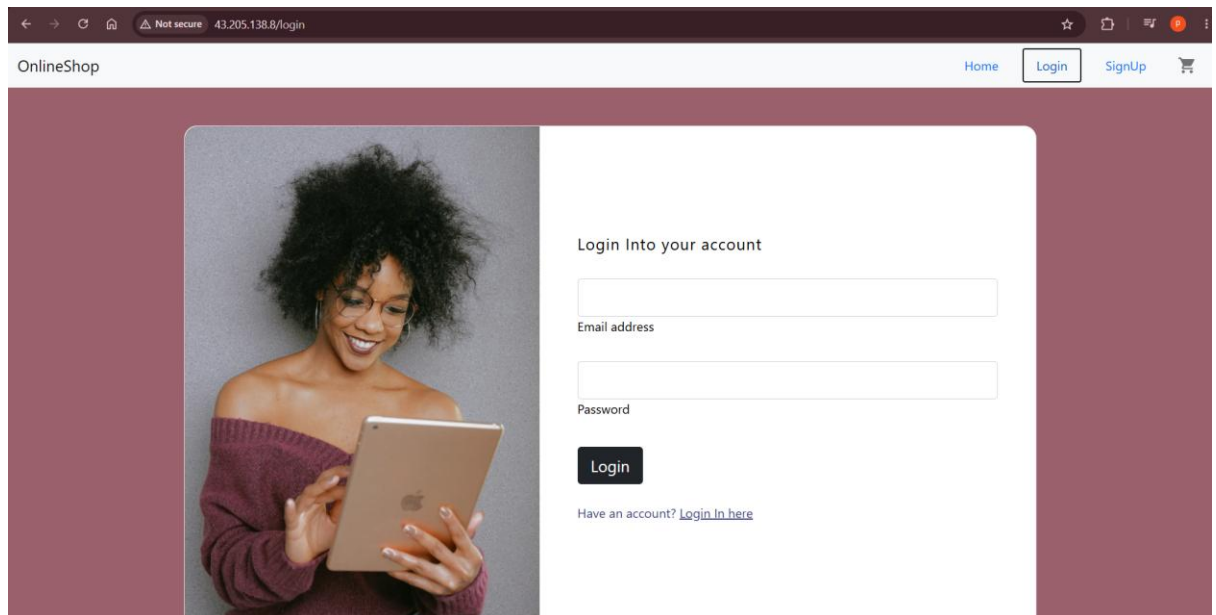
Owner ID

Launch time

Security groups

i-077163f7c61bc2ca7 (devops-build-server)					
-	sgr-06ea214220cd66ebc	3001	TCP	0.0.0.0/0	launch-wizard-7
-	sgr-0fed43e710230b960	80	TCP	0.0.0.0/0	launch-wizard-7
-	sgr-004fd784545910298	22	TCP	103.212.145.248/32	launch-wizard-7
-	sgr-0093e268cb639f11d	443	TCP	0.0.0.0/0	launch-wizard-7

7. Final Application Running in Browser



Monitoring:

